
Introduction to Artificial Intelligence

Jinshuo Li

Shanghai Jiao Tong University

January 18, 2026

Contents

Preface

1	Basics	1
1.1	Language and Packages	1
1.2	Introduction to Tensor	2
1.3	Data Preparation	4
1.3.1	Data Reading	4
1.3.2	Handling Missing Values	4
1.3.3	Convert the Type to Tensor	5
1.4	Some Linear Algebra	6
1.4.1	Vectors	6
1.4.2	Matrix	6
1.4.3	Tensor	7
1.5	Some Analysis	9
1.5.1	Derivation	9
1.5.2	Gradient	9
1.5.3	Chain Rule	10
1.5.4	Automatic Differentiation	10
1.5.5	Probability	11
1.6	More Informations	12
2	Linear Regression	13
2.1	Basic Concepts	13
2.1.1	Linear Model	13
2.1.2	Loss Function	13
2.1.3	Analytical Solution for Linear Regression	14
2.1.4	Random Gradient Descent	14
2.1.5	Normal Distribution and Linear Regression	15
2.1.6	Neural Network Perspective	16
2.2	How to Achieve Linear Regression in Practice	17
2.3	How to Make Things Simpler	19
2.4	Softmax Regression	21
2.4.1	Model Definition	21
2.4.2	MNIST	23
3	Multilayer Perceptrons	29

Preface

Standing at the beginning of 2026, looking back at the tumultuous development of Artificial Intelligence over the past few years, one cannot help but feel a sense of awe. From the initial explosion of generative models to the ubiquity of intelligent agents today, AI has fundamentally reshaped the way we research, work, and live.

However, amidst the dazzling array of applications and the rapid iteration of tools, the underlying principles often remain obscured by the hype. As a student at Shanghai Jiao Tong University, I have spent much of my time navigating through this fog, attempting to connect the classical theories of machine learning with the emergent behaviors of modern large models.

This document is not merely a collection of technical notes; it is an attempt to construct a coherent narrative of AI knowledge as I understand it. It begins with the fundamental mathematics and algorithms, traverses through the architectures that defined the deep learning era, and arrives at the frontier of current research.

My hope is that this work will serve as a roadmap for those who, like me, are eager to look beyond the "black box" and understand the elegant logic that drives the intelligence of our time.

Writing is a process of learning. All errors and omissions are my own, and I welcome any corrections.

Jinshuo Li
Shanghai Jiao Tong University
2026.1.6, in Beijing

Chapter 1

Basics

This is the first chapter of the document. Here we will cover the basic concepts and foundational information necessary for understanding the subsequent chapters.

1.1 Language and Packages

To follow the trends in research of artificial intelligence, we will use **Python** as the programming language throughout this book. Python is widely adopted in the AI community due to its simplicity and the vast array of libraries available for machine learning, data analysis, and scientific computing.

To be specific, I would like to inform the readers that we use **3.11.9** version of Python in this book at the time of January 18, 2026. You can download it from the official Python website: <https://www.python.org/downloads/release/python-3119/>. This version is stable and includes the latest features and improvements, while maintaining compatibility with most libraries used in AI research.

Also, we will use several popular Python libraries throughout the book, including but not limited to:

- **NumPy**: For numerical computations and array manipulations.
- **Pandas**: For data manipulation and analysis.
- **Matplotlib** and **Seaborn**: For data visualization.
- **Pytorch** and **TensorFlow**: For building and training machine learning models.

We have to say that this should be a continuously updating list as new libraries and tools are developed in the fast-evolving field of AI. I encourage readers to stay updated with the latest developments and consider exploring additional libraries that may be relevant to their specific interests and projects.

1.2 Introduction to Tensor

A **tensor** is a mathematical object that generalizes the concepts of scalars, vectors, and matrices to higher dimensions. Tensors are fundamental in representing data in various forms, especially in deep learning frameworks like TensorFlow and PyTorch. You can simply think of tensors as multi-dimensional arrays. It is similar to **ndarray** in **NumPy**, which is a powerful library for numerical computing in Python. Just like ndarrays, tensors can hold data of various types (e.g., integers, floats) and can be manipulated using a variety of operations.

To follow the trends, we use **PyTorch** tensors in this book. PyTorch is a popular deep learning framework that provides efficient tensor operations and automatic differentiation, making it easier to build and train neural networks. The most remarkable feature of PyTorch tensors is their ability to leverage GPU acceleration for faster computations, which is crucial for training large-scale machine learning models.

Python Code: Basic tensors usage in PyTorch

```

1 import torch
2
3 # Arange tensor
4 tensor_x = torch.arange(0, 10, step=2)
5 tensor_y = torch.zeros((2,3,4))
6 tensor_w = torch.rand((2,2))
7
8 # Creating a tensor
9 tensor_a = torch.tensor([[1, 2], [3, 4]])
10
11 # Properties of the tensor
12 print("Shape:", tensor_a.shape)
13 print("Size:", tensor_a.numel())
14 print("Total:", tensor_a.sum().item())
15 # tensor.item() to get the value as a standard Python number
16
17 # Tensor operations (Different to Linear Algebra)
18 tensor_b = tensor_a.reshape(4, 1)
19 tensor_c_group = (tensor_a + tensor_b, tensor_a - tensor_b, tensor_a * tensor_b,
20                  tensor_a / tensor_b, tensor_a ** tensor_b)
21 tensor_d = (tensor_b == tensor_a) # Element-wise operations, returns a new
22   tensor stuffed with boolean values.
23
24 # Tensor Concatentation
25 tensor_d = torch.cat((tensor_a, tensor_a), dim=0) # Concatenate after rows
26 tensor_e = torch.cat((tensor_a, tensor_a), dim=1) # Concatenate after columns
27
28 # Tensor Broadcasting
29 tensor_f = tensor_a + torch.tensor([1, 2])
30
31 # Tensor Indexing and Slicing
32 element = tensor_a[0, 1] # Accessing a single element
33 row = tensor_a[1, :] # Accessing a row
34 column = tensor_a[:, 0] # Accessing a column
35 sub_tensor = tensor_a[0:2, 0:2] # Slicing
36
37 # Tensor Type Conversion
38 tensor_h = tensor_a.int() # Convert to int type, float type is the same.
39 ndarray_from_tensor = tensor_a.numpy() # Convert to NumPy ndarray

```

Remark 1.2.1

Here are a few additional notes for the code above on tensors in PyTorch:

- Broadcasting addition as an example, adds `[1, 2]` to each row of `tensor_a`. It's a process of expanding (duplicating) the smaller tensor to match the shape of the larger tensor during arithmetic operations.
- PyTorch tensors support a wide range of operations, including mathematical functions, linear algebra operations, and more.
- Tensors can be moved between CPU and GPU memory for efficient computation using the `'to(device)'` method.
- PyTorch provides automatic differentiation capabilities through the `'autograd'` module, which is essential for training neural networks.

We will introduce more details about tensors and their applications in deep learning as we progress through the book. Understanding tensors is crucial for effectively working with machine learning models and implementing various algorithms.

We encourage readers to experiment with tensors in PyTorch and explore their capabilities further.

More information about PyTorch tensors can be found in the official documentation: <https://pytorch.org/docs/stable/tensors.html>.

1.3 Data Preparation

Data preparation is a crucial step in the machine learning pipeline, as the quality and structure of the data can significantly impact the performance of the models. In this section, we will discuss some common techniques for preparing data for machine learning tasks.

1.3.1 Data Reading

For example, if we want to read a CSV file containing our dataset, we can use the Pandas library in Python: Firstly, we need to create a sample CSV file named `house_tiny.csv` in the `data` directory.

Python Code: Read files

```
1 import os
2
3 os.makedirs(os.path.join('.', 'data'), exist_ok=True)
4 data_file = os.path.join('.', 'data', 'house_tiny.csv')
5 with open(data_file, 'w') as f:
6     f.write('NumRooms,Alley,Price\n') # Header for each column
7     f.write('NA,Pave,127500\n')      # Each row represents a data sample
8     f.write('2,NA,106000\n')
9     f.write('4,NA,178100\n')
10    f.write('NA,NA,140000\n')
```

Now, we can read the CSV file using Pandas:

Python Code: Read CSV file using Pandas

```
1 import pandas as pd
2
3 data = pd.read_csv(data_file)
4 print(data)
```

The output will be:

	NumRooms	Alley	Price
0	NA	Pave	127500
1	2	NA	106000
2	4	NA	178100
3	NA	NA	140000

1.3.2 Handling Missing Values

In real-world datasets, it is common to encounter missing values. There are several strategies to handle missing data, including:

- **Removing rows or columns:** If a row or column has a significant amount of missing data, it may be best to remove it entirely.
- **Imputation:** Filling in missing values with statistical measures such as the mean, median, or mode of the respective column.
- **Using algorithms that support missing values:** Some machine learning algorithms can handle missing data without any preprocessing.

Here we consider to use mean imputation for numerical columns and mode imputation for categorical columns.

Python Code: Handling Missing Values

```
1 inputs, outputs = data.iloc[:, 0:2], data.iloc[:, 2]
2 inputs = inputs.fillna(inputs.mean())
3 print(inputs)
```

This way we have handled the missing values in the **inputs**. The output will be:

	NumRooms	Alley
0	3.0	Pave
1	2.0	NaN
2	4.0	NaN
3	3.0	NaN

We can also view **NaN** as a category in the **Alley** column. Then we can transform the categorical variable into the form of:

Python Code: Categorical variable encoding

```
1 inputs = pd.get_dummies(inputs, dummy_na=True)
2 print(inputs)
```

The output will be:

	NumRooms	Alley_NA	Alley_Pave
0	3.0	0	1
1	2.0	1	0
2	4.0	1	0
3	3.0	1	0

1.3.3 Convert the Type to Tensor

After handling missing values and encoding categorical variables, we can convert the data into tensors for further processing in machine learning models.

Python Code: Convert DataFrame to Tensors

```
1 import torch
2
3 X = torch.tensor(inputs.to_numpy(), dtype=torch.float32)
4 Y = torch.tensor(outputs.to_numpy(), dtype=torch.float32)
5
6 print(X)
7 print(Y)
```

Then we have successfully converted the prepared data into PyTorch tensors, which can be used for training machine learning models.

1.4 Some Linear Algebra

In this section, we will review some fundamental concepts of linear algebra that are essential for understanding machine learning algorithms.

1.4.1 Vectors

A **vector** is an ordered collection of numbers that can be represented as a point in a multi-dimensional space. Vectors can be added together and multiplied by scalars, and they are often used to represent data points, features, or weights in machine learning. In **PyTorch**, vectors are represented as one-dimensional tensors.

Python Code: i

```
1 import torch
2
3 vector_a = torch.tensor([1, 2, 3])
4 vector_b = torch.tensor([4, 5, 6])
```

We can visit each element of the vector using indexing:

Python Code: Vector Indexing

```
1 first_element = vector_a[0]
2 print("First element of vector_a:", first_element)
```

Just like arrays in NumPy, PyTorch tensors support various operations on vectors, such as addition, subtraction, dot product, and more.

Dot Product

The **dot product** (or scalar product) of two vectors is a mathematical operation that takes two equal-length sequences of numbers and returns a single number. The dot product is calculated by multiplying corresponding elements of the vectors and summing the results.

Python Code: Dot Product

```
1 dot_product = torch.dot(vector_a, vector_b)
2 print("Dot product of vector_a and vector_b:", dot_product)
```

1.4.2 Matrix

A **matrix** is a two-dimensional array of numbers arranged in rows and columns. Matrices are used to represent data, transformations, and relationships between variables in machine learning. In **PyTorch**, matrices are represented as two-dimensional tensors.

Python Code: i

```
1 import torch
2
3 matrix_a = torch.tensor([[1, 2, 3], [4, 5, 6]])
4 matrix_b = torch.tensor([[7, 8, 9], [10, 11, 12]])
```

We can access elements of the matrix using row and column indices:

Python Code: Matrix Indexing

```
1 element_1_2 = matrix_a[1, 2] # Accessing element in the 2nd row and 3rd column
2 print("Element at (1, 2) of matrix_a:", element_1_2)
```

We can perform matrix transposition, multiplication, and other operations using PyTorch functions.

Python Code: Operations on Matrices

```
1 matrix_c = matrix_a.T # Transpose of matrix_a
```

1.4.3 Tensor

A **tensor** is a multi-dimensional array that generalizes the concepts of scalars (0D), vectors (1D), and matrices (2D) to higher dimensions. Tensors can have any number of dimensions and are used to represent complex data structures in machine learning and deep learning. In **PyTorch**, tensors are the primary data structure for storing and manipulating data.

Python Code: i

```
1 import torch
2
3 tensor_a = torch.arange(24).reshape(2, 3, 4) # A 3D tensor with shape (2, 3, 4)
```

We can access elements of the tensor using multiple indices corresponding to each dimension:

Python Code: Tensor Indexing

```
1 element_1_2_3 = tensor_a[1, 2, 3] # Accessing element in the 2nd block, 3rd row,
    and 4th column
2 print("Element at (1, 2, 3) of tensor_a:", element_1_2_3)
```

We can clone tensors to create a copy of the tensor:

Python Code: Cloning Tensors

```
1 tensor_b = tensor_a.clone()
```

Remark 1.4.1

The `.clone()` method is a deepcopy operation that creates a new tensor with its own memory allocation. This means that any modifications made to the cloned tensor will not affect the original tensor, and vice versa. This is particularly useful when you want to preserve the original data while performing operations or transformations on a copy of the tensor.

We can also perform various operations on tensors, such as addition, multiplication, and more. We have to know that all the binary operations between two tensors in PyTorch are element-wise operations. The multiplication between two tensors is not the matrix multiplication but the element-wise multiplication. For matrix multiplication, we have to use the `torch.matmul()` function or the `@` operator.

Dimensionality Reduction

Dimensionality reduction is a technique used to reduce the number of features or dimensions in a dataset while preserving its essential characteristics. This is particularly useful in machine learning, where high-dimensional data can lead to overfitting and increased computational complexity.

One common method is summing over specific dimensions of a tensor. In PyTorch, we can use the `torch.sum()` function to achieve this.

Python Code: Dimensionality Reduction by Summation

```
1 import torch
2
3 tensor_a = torch.arange(24).reshape(2, 3, 4) # A 3D tensor with shape (2, 3, 4)
4 reduced_tensor_a = torch.sum(tensor_a, dim=0) # Summing over the first dimension
5 reduced_tensor_b = torch.sum(tensor_a, dim=1) # Summing over the second
   dimension
6 reduced_tensor_c = torch.sum(tensor_a, dim=2) # Summing over the third dimension
```

We can also use the `keepdim` parameter to retain the reduced dimensions with size one, which can be useful for broadcasting in subsequent operations.

Python Code: Dimensionality Reduction with keepdim

```
1 reduced_tensor_d = torch.sum(tensor_a, dim=1, keepdim=True) # Summing over the
   second dimension and keeping the dimension
```

Norm

The **norm** of a tensor is a measure of its magnitude or length. In PyTorch, we can compute various types of norms using the `torch.norm()` function. The most commonly used norms are the L_1 norm (sum of absolute values) and the L_2 norm (Euclidean norm).

More specifically, for a tensor A , the L_1 norm is defined as:

$$\|A\|_1 = \sum_{i,j,k} |a_{i,j,k}|$$

and the L_2 norm is defined as:

$$\|A\|_2 = \sqrt{\sum_{i,j,k} a_{i,j,k}^2}$$

Here is an example of computing the L_1 and L_2 norms of a tensor in PyTorch:

Python Code: Computing Norms of Tensors

```
1 import torch
2 tensor_a = torch.tensor([[1.0, -2.0, 3.0], [-4.0, 5.0, -6.0]])
3 l1_norm = torch.norm(tensor_a, p=1) # L1 norm
4 l2_norm = torch.norm(tensor_a, p=2) # L2 norm
```

1.5 Some Analysis

In this section, we will review some fundamental concepts of analysis that are essential for understanding machine learning algorithms.

About 2500 years ago, the ancient Greek mathematician Eudoxus of Cnidus developed the method of exhaustion, which laid the groundwork for integral calculus. Later, in the 17th century, Isaac Newton and Gottfried Wilhelm Leibniz independently developed the formal framework of calculus, introducing concepts such as limits, derivatives, and integrals.

Just as we can see, the most important question in analysis, is **optimization**, which is the process of finding the best solution from a set of possible solutions. In machine learning, optimization is used to minimize a loss function, which measures the difference between the predicted output and the actual output.

1.5.1 Derivation

In monadic calculus, the **derivative** of a function measures how the function changes as its input changes. It is a fundamental concept in optimization, as it provides information about the slope of the function at a given point.

$$f'(x) = \lim_{\Delta \rightarrow 0} \frac{f(x + \Delta) - f(x)}{\Delta}$$

In multivariate calculus, the **partial derivative** of a function with respect to one of its variables measures how the function changes as that variable changes, while keeping the other variables constant.

$$\frac{\partial f}{\partial x_i} = \lim_{\Delta \rightarrow 0} \frac{f(x_1, \dots, x_i + \Delta, \dots, x_n) - f(x_1, \dots, x_i, \dots, x_n)}{\Delta}$$

1.5.2 Gradient

The **gradient** of a multivariate function is a vector that contains all of its partial derivatives. It points in the direction of the steepest ascent of the function and is used in optimization algorithms to find the minimum or maximum of a function.

$$\nabla_x f(x) = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n} \right)^T$$

For gradient, we have following properties:

- $\forall A \in \mathbb{R}^{m \times n}, \nabla_x Ax = A^T$
- $\forall A \in \mathbb{R}^{n \times m}, \nabla_x x^T A = A$
- $\forall A \in \mathbb{R}^{n \times n}, \nabla_x x^T Ax = (A + A^T)x$
- $\nabla_x \|x\|^2 = \nabla_x (x^T x) = 2x$

These properties are useful in deriving gradients for various functions commonly encountered in machine learning and optimization problems.

1.5.3 Chain Rule

The **chain rule** is a fundamental theorem in calculus that allows us to compute the derivative of a composite function. If we have two functions f and g , the chain rule states that the derivative of the composite function $h(x) = f(g(x))$ is given by:

$$h'(x) = f'(g(x)) \cdot g'(x)$$

In the context of multivariate functions, the chain rule can be extended to compute the gradient of a composite function. If $f : \mathbb{R}^m \rightarrow \mathbb{R}$ and $g : \mathbb{R}^n \rightarrow \mathbb{R}^m$, then the gradient of the composite function $h(x) = f(g(x))$ is given by:

$$\nabla_x h(x) = J_g(x)^T \nabla_y f(y) \Big|_{y=g(x)}$$

where $J_g(x)$ is the Jacobian matrix of g at x .

1.5.4 Automatic Differentiation

In machine learning, we often need to compute gradients of complex functions with respect to their inputs. **Automatic differentiation** is a technique that allows us to compute these gradients efficiently and accurately. It works by breaking down the computation of a function into a series of elementary operations, each of which has a known derivative. By applying the chain rule systematically, we can compute the gradient of the entire function.

Here is a simple example of automatic differentiation using **PyTorch**:

Python Code: Automatic Differentiation in PyTorch

```

1 import torch
2
3 # Create a tensor with gradient tracking enabled
4 x = torch.arange(4.0, requires_grad=True)
5
6 # Define a function of x
7 y = 2 * torch.dot(x, x)
8
9 # Compute the gradient of y with respect to x
10 y.backward()
11
12 # Print the computed gradient, stored in x.grad as a property of the tensor x
13 print("Gradient of y with respect to x:", x.grad)

```

The output will be:

Gradient of y with respect to x: tensor([0., 4., 8., 12.])

This indicates that the gradient of y with respect to x is $[0, 4, 8, 12]$, which corresponds to the derivative of the function $y = 2(x_0^2 + x_1^2 + x_2^2 + x_3^2)$ evaluated at $x = [0, 1, 2, 3]$.

If we want to calculate the result of a function with two or more variables, and we only want to calculate the gradient with respect to one of them, we can use the `.detach()` parameter in the process to retain the computation graph for further gradient calculations.

Python Code: Automatic Differentiation with Multiple Variables

```

1 import torch
2
3 # Create tensors with gradient tracking enabled
4 x = torch.arange(4.0, requires_grad=True)
5 y = torch.arange(1.0, 5.0, requires_grad=True)
6
7 # Define a function of x and y
8 # Detach y to prevent gradient tracking for y
9 z = torch.dot(x, y.detach())
10
11 # Compute the gradient of z with respect to x
12 z.backward()
13
14 # Print the computed gradient, stored in x.grad as a property of the tensor x
15 print("Gradient of z with respect to x:", x.grad)

```

Then we can get the gradient of z with respect to x only.

1.5.5 Probability

Simply speaking, machine learning is all about making predictions based on data. Since data is often noisy and uncertain, we need to use probability theory to model this uncertainty and make informed predictions. Probability provides a mathematical framework for quantifying uncertainty and making decisions based on incomplete information.

We shall introduce some basic concepts of probability that are relevant to machine learning.

Definition 1.5.1

The Axioms of Probability In a given sample space S , the probability function $P(A)$ satisfies the following axioms:

- Non-negativity: For any event A , $P(A) \geq 0$.
- Normalization: The probability of the entire sample space is 1, i.e., $P(S) = 1$.
- Additivity: For any countable sequence of mutually exclusive events $A_1, A_2, A_3, \dots$, the probability of their union is equal to the sum of their individual probabilities:

$$P\left(\bigcup_{i=1}^{\infty} A_i\right) = \sum_{i=1}^{\infty} P(A_i)$$

Definition 1.5.2

Conditional Probability The conditional probability of an event A given that another event B has occurred is defined as:

$$P(A|B) = \frac{P(A \cap B)}{P(B)}, \quad \text{if } P(B) > 0$$

Definition 1.5.3

Bayes' Theorem Bayes' theorem relates the conditional probabilities of two events A and B :

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}, \quad \text{if } P(B) > 0$$

Definition 1.5.4

Marginal Probability The marginal probability of an event A is the total probability of A occurring, regardless of the occurrence of other events. It can be computed by summing or integrating over the probabilities of all possible outcomes that include A :

$$P(A) = \sum_i P(A \cap B_i) \quad \text{or} \quad P(A) = \int P(A \cap B) dB$$

where B_i represents all possible events that can occur alongside A .

Definition 1.5.5

Expectation and Variance The expectation (or expected value) of a random variable X is a measure of its central tendency and is defined as:

$$E[X] = \sum_x x \cdot P(X = x) \quad (\text{for discrete random variables})$$

$$E[X] = \int x \cdot f_X(x) dx \quad (\text{for continuous random variables})$$

The variance of a random variable X measures the spread of its values around the expectation and is defined as:

$$\text{Var}(X) = E[(X - E[X])^2] = E[X^2] - (E[X])^2$$

1.6 More Informations

Here, we provide some ways to find special properties.

Python Code: Finding Special Properties for Specific Modules

```
1 import torch
2
3 # For example, to find all the properties and methods of the torch.nn module
4 print(dir(torch.nn))
```

Python Code: Finding Documentation for Specific Variables

```
1 import torch
2
3 x = torch.arange(4.0, requires_grad=True)
4 help(torch.x)
```

Then we can get the documentation for the variable `x`.

Chapter 2

Linear Regression

Regression is a fundamental concept that can modelling the relationship between a dependent variable and one or more independent variables. Linear regression is one of the simplest and most widely used regression techniques. It assumes a linear relationship between the input features and the output variable.

2.1 Basic Concepts

2.1.1 Linear Model

Traditional linear model can be represented as:

$$y = w^T x + b$$

where y is the predicted output, x is the input feature vector, w is the weight matrix, and b is the bias term.

More rigorously speaking, the formula above is a kind of **affine transformation**, which is a linear transformation followed by a translation. In this case, the linear transformation is represented by the weight matrix w , and the translation is represented by the bias term b .

2.1.2 Loss Function

The loss function quantifies the difference between the predicted output and the actual output. In linear regression, the most commonly used loss function is defined as:

$$l^{(i)} = \frac{1}{2}(y^{(i)} - \hat{y}^{(i)})^2$$

where $l^{(i)}$ is the loss for the i -th training example, $y^{(i)}$ is the actual output, and $\hat{y}^{(i)}$ is the predicted output.

To measure the overall performance of the model on the entire training dataset, we can use the Mean Squared Error (MSE) loss function, which is defined as:

$$L = \frac{1}{m} \sum_{i=1}^m l^{(i)} = \frac{1}{2m} \sum_{i=1}^m (y^{(i)} - \hat{y}^{(i)})^2$$

where L is the overall loss, and m is the number of training examples.

The overall purpose is to find the optimal parameters w and b that minimize the loss function L .

2.1.3 Analytical Solution for Linear Regression

Unlike other machine learning algorithms that require iterative optimization techniques, linear regression has a closed-form solution. The optimal parameters w and b can be computed directly using the following formulas:

$$w = (X^T X)^{-1} X^T y$$

$$b = \bar{y} - w^T \bar{x}$$

where X is the input feature matrix, y is the output vector, $\bar{y}$ is the mean of the output values, and $\bar{x}$ is the mean of the input features.

Analytical solution is computationally efficient for small to medium-sized datasets. However, for large datasets or high-dimensional feature spaces, iterative optimization techniques such as Gradient Descent are often preferred.

To better simulate the process of finding optimal parameters, we will introduce Gradient Descent in the next section.

2.1.4 Random Gradient Descent

Eventhough we have an analytical solution for linear regression, it is still beneficial to understand and implement Gradient Descent, as it is a widely used optimization technique in machine learning. In future cases, we will need to use Gradient Descent to optimize more complex models where analytical solutions are not feasible.

The simplest version of Gradient Descent is to calculate the loss function on the entire training dataset and update the parameters accordingly. This is known as Batch Gradient Descent. However, for large datasets, this approach can be computationally expensive. Instead, we can use minibatch stochastic gradient descent (SGD), which updates the parameters using a small random subset of the training data (minibatch) at each iteration.

The process can be expressed as follows:

1. Randomly select a minibatch of training examples from the dataset.
2. Compute the predicted outputs for the selected minibatch using the current parameters w and b .
3. Calculate the gradients of the loss function with respect to w and b using the selected minibatch.
4. Update the parameters w and b using the computed gradients and a predefined learning rate α :

$$w := w - \alpha \frac{\partial L}{\partial w} \quad b := b - \alpha \frac{\partial L}{\partial b}$$

5. Repeat steps 1-4 for a predefined number of iterations or until convergence.
6. Return the optimized parameters w and b .

Using minibatch SGD allows us to continuously convergent to the optimal parameters while reducing the computationally cost per iteration. But we can't actually guarantee that the parameters will converge to the exact optimal values as in the analytical solution. However, with a proper learning rate and sufficient iterations, we can achieve satisfactory results.

Linear regression with Gradient Descent provides a solid foundation for understanding optimization techniques in machine learning, which can be extended to more complex models in future chapters.

During the process, we recommend the reader to use **PyTorch** or **TensorFlow** to implement the algorithm, as these frameworks provide efficient tools for automatic differentiation and optimization.

2.1.5 Normal Distribution and Linear Regression

Now let's go back to the loss function we defined earlier:

$$l^{(i)} = \frac{1}{2}(y^{(i)} - \hat{y}^{(i)})^2$$

Why do we use this specific form of loss function in linear regression? The answer lies in the assumption that the errors (the differences between the actual outputs and the predicted outputs) follow a normal distribution, which is a common assumption in statistical modeling.

So first, let's take a look at what is a normal distribution.

Definition 2.1.1

A normal distribution is a continuous probability distribution characterized by its mean μ and standard deviation σ . The probability density function (PDF) of a normal distribution is given by:

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

Definition 2.1.2

In statistics, the likelihood function (often simply called the likelihood) measures the plausibility of a set of model parameters given some observed data. It is defined as the probability (or probability density) of observing the data, expressed as a function of the parameters.

Mathematically, if we have a set of observed data points $D = \{x_1, x_2, \dots, x_n\}$ and a statistical model with parameters θ , the likelihood function $L(\theta; D)$ is given by:

$$L(\theta; D) = P(D|\theta)$$

And our assumption can be written as:

$$y = w^T x + b + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \sigma^2)$$

where ϵ represents the error term, which follows a normal distribution with mean 0 and variance σ^2 . Now we can derive the loss function from this assumption.

Given a training dataset with m examples, the likelihood function for the entire dataset can be expressed as the product of the individual likelihoods for each training example:

$$L(w, b) = \prod_{i=1}^m P(y^{(i)}|x^{(i)}; w, b)$$

Since the errors follow a normal distribution, we can express the conditional probability $P(y^{(i)}|x^{(i)}; w, b)$ as:

$$P(y^{(i)}|x^{(i)}; w, b) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(y^{(i)} - (w^T x^{(i)} + b))^2}{2\sigma^2}}$$

Substituting this into the likelihood function, we get:

$$L(w, b) = \prod_{i=1}^m \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(y^{(i)} - (w^T x^{(i)} + b))^2}{2\sigma^2}}$$

To simplify the optimization process, we often work with the log-likelihood function instead of the likelihood function itself. Taking the natural logarithm of the likelihood function, we have:

$$\ln L(w, b) = \sum_{i=1}^m \left(-\ln(\sigma\sqrt{2\pi}) - \frac{(y^{(i)} - (w^T x^{(i)} + b))^2}{2\sigma^2} \right)$$

Maximizing the log-likelihood function is equivalent to minimizing the negative log-likelihood. Ignoring the constant term $-\ln(\sigma\sqrt{2\pi})$, we can focus on minimizing the following expression:

$$\sum_{i=1}^m \frac{(y^{(i)} - (w^T x^{(i)} + b))^2}{2\sigma^2}$$

Since σ^2 is a constant, minimizing this expression is equivalent to minimizing:

$$\sum_{i=1}^m \frac{1}{2} (y^{(i)} - (w^T x^{(i)} + b))^2$$

This is exactly the loss function we defined earlier for linear regression. Therefore, by assuming that the errors follow a normal distribution, we arrive at the conclusion that minimizing the Mean Squared Error (MSE) loss function is equivalent to maximizing the likelihood of the observed data given the model parameters.

2.1.6 Neural Network Perspective

All the content shown above can be easily extended to neural networks. In fact, a linear regression model can be seen as a simple neural network with no hidden layers and a linear activation function.

When we extend the linear model to a neural network, we introduce multiple layers of neurons, each with its own set of weights and biases. The output of each layer is passed through an activation function, which introduces non-linearity into the model. This allows the neural network to learn more complex relationships between the input features and the output variable.

We can express the neural network model as the following figure:

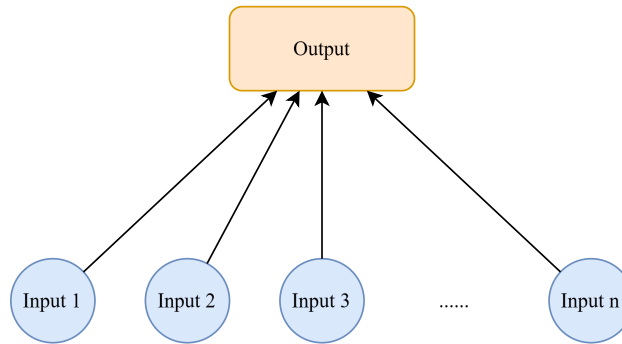


Figure 2.1: Neural Network Perspective of Linear Regression

We can see that the linear regression model is equivalent to a neural network with a single neuron, where the weights and bias correspond to the parameters of the linear model. The loss function remains the same, as we still aim to minimize the Mean Squared Error (MSE) between the predicted outputs and the actual outputs.

For the neural network of linear regression, all the inputs are directly connected to the output neuron without any hidden layers. The output neuron performs a weighted sum of the inputs, adds a bias term, and produces the final output. We call this type of neural network a **fully-connected layer** or **dense layer**.

Some Biology

In biological neural networks, neurons are interconnected cells that transmit information through electrical and chemical signals. Each neuron consists of a cell body (soma), dendrites (input branches), and an axon (output branch). Dendrites receive signals from other neurons, which are then processed in the cell body. If the signal is strong enough, the neuron generates an action potential (electrical impulse) that travels down the axon to communicate with other neurons.

But nowadays, artificial neural networks are only loosely inspired by biological neural networks. While they share some similarities in terms of structure and function, artificial neural networks are simplified mathematical models that do not capture the full complexity of biological neurons, just like the Russell and Norvig said in their book *Artificial Intelligence: A Modern Approach*.

What we have shown above is just a simple example of how linear regression can be viewed from a neural network perspective. As we delve deeper into the field of machine learning and neural networks, we will explore more complex architectures and techniques that build upon this foundational understanding.

2.2 How to Achieve Linear Regression in Practice

After we know the key idea of linear regression, we can implement it in practice using popular machine learning libraries **PyTorch**.

Here is a simple example of how to implement linear regression using PyTorch:

Python Code: Libraries

```
1 import random
2 import torch
```

To make things simpler, we will generate some synthetic data for training the linear regression model.

Python Code: Data Generation

```
1 # Generate synthetic data
2 def generate_data(w, b, num_examples):
3     X = torch.normal(0, 1, (num_examples, len(w)))
4     y = torch.matmul(X, w) + b
5     y += torch.normal(0, 0.01, y.shape)
6     return X, y.reshape((-1, 1))
7
8 true_w = torch.tensor([2.0, -3.4])
9 true_b = 4.2
10 features, labels = generate_data(true_w, true_b, 1000)
```

Now we can define the linear regression model using PyTorch's built-in modules. Have a review, we need to read the data, randomly select minibatches, define the model, define the loss function, and optimize the model parameters using Gradient Descent. Now we need to implement these steps in code.

Python Code: Random Minibatch Sampling

```
1 # Define a function to get minibatches
2 def data_iter(batch_size, features, labels):
3     num_examples = len(features)
4     indices = list(range(num_examples))
5     random.shuffle(indices)
6     for i in range(0, num_examples, batch_size):
7         batch_indices = torch.tensor(
8             indices[i: min(i + batch_size, num_examples)])
9         yield features[batch_indices], labels[batch_indices]
```

Why we use minibatches? Because using the entire dataset to compute the gradients can be computationally expensive, especially for large datasets. Minibatch SGD allows us to update the model parameters more

frequently. Also, take advantage of the **GPU** acceleration, which can significantly speed up the training process.

Now let's define the linear regression model, first we need to initialize the model parameters (weights and bias):

Python Code: Model Definition

```
1 w = torch.normal(0, 0.01, size=(2, 1), requires_grad=True)
2 b = torch.zeros(1, requires_grad=True)
```

Our next step is to construct the model's forward pass, which computes the predicted outputs given the input features, as well as defining the loss function and the optimization algorithm.

Python Code: Main Components

```
1 # Define the linear regression model
2 def linreg(X, w, b):
3     return torch.matmul(X, w) + b
4
5 # Define the Mean Squared Error loss function
6 def mse_loss(y_hat, y):
7     return ((y_hat - y.reshape(y_hat.shape)) ** 2) / 2
8
9 # Define the SGD optimization algorithm
10 def sgd(params, lr, batch_size):
11     with torch.no_grad():
12         for param in params:
13             param -= lr * param.grad / batch_size
14             param.grad.zero_()
```

Now we can train the linear regression model using the defined components. We will iterate over the training data for a specified number of epochs, updating the model parameters using minibatch SGD.

Python Code: Training Loop

```
1 # Training loop
2 lr = 0.03
3 num_epochs = 3
4 net = linreg
5 loss = mse_loss
6
7 # Main loop
8 for epoch in range(num_epochs):
9     for X, y in data_iter(10, features, labels):
10         l = loss(net(X, w, b), y)
11         l.sum().backward()
12         sgd([w, b], lr, batch_size=10)
13     with torch.no_grad():
14         train_l = loss(net(features, w, b), labels)
15         print(f'epoch {epoch + 1}, loss {float(train_l.mean()):f}')
```

In the process of `l.sum().backward()`, PyTorch automatically computes the gradients of the loss with respect to the model parameters w and b using backpropagation. This is one of the key features of PyTorch that simplifies the implementation of machine learning algorithms.

After training the model, we can evaluate its performance by comparing the learned parameters w and b with the true parameters used to generate the synthetic data.

Python Code: Model Evaluation

```
1 print(f'Learned weights: {w.reshape(true_w.shape)}')
2 print(f'Learned bias: {b}')
```

To summarize, we have implemented a linear regression model using PyTorch, demonstrating the key steps involved in training the model using minibatch stochastic gradient descent.

We can summarize it step by step as follows:

1. Input the training data (features X and labels y).
2. Forward pass: Compute the predicted outputs $\hat{y}$ using the current model parameters w and b .
3. Loss computation: Calculate the loss L using the Mean Squared Error (MSE) loss function.
4. Backward pass: Compute the gradients of the loss with respect to the model parameters w and b using backpropagation.
5. Parameter update: Update the model parameters w and b using the computed gradients and the learning rate α with the SGD optimization algorithm.
6. Repeat steps 2-5 for a predefined number of epochs or until convergence.

You can run the complete code in a Python environment with PyTorch installed to see the results. We provide the complete code in the folder for your convenience.

2.3 How to Make Things Simpler

We will introduce some predefined modules in PyTorch that can help us simplify the implementation of linear regression.

Just like what we have done before, we still need to define the model, loss function, and optimization algorithm. But this time, we can use PyTorch's built-in modules to simplify these steps.

Firstly, we still need to generate the synthetic data as before.

Python Code: Data Generation

```
1 import random
2 import torch
3 from torch import nn
4
5 true_w = torch.tensor([2.0, -3.4])
6 true_b = 4.2
7 def generate_data(w, b, num_examples):
8     X = torch.normal(0, 1, (num_examples, len(w)))
9     y = torch.matmul(X, w) + b
10    y += torch.normal(0, 0.01, y.shape)
11    return X, y.reshape((-1, 1))
12
13 features, labels = generate_data(true_w, true_b, 1000)
```

Then, how can we read the data in minibatches? We can use PyTorch's built-in `DataLoader` module to handle the batching and shuffling of the data for us.

Python Code: DataLoader

```
1 def load_array(data_arrays, batch_size, is_train=True):
2     dataset = data.TensorDataset(*data_arrays)
3     return data.DataLoader(dataset, batch_size, shuffle=is_train)
4
5 batch_size = 10
6 data_iter = load_array((features, labels), batch_size)
```

Next, we can define the linear regression model using PyTorch's `nn.Linear` module, which automatically handles the initialization of weights and biases for us.

Python Code: Model Definition

```
1 # Define the linear regression model using nn.Linear
2 net = nn.Linear(2, 1)
```

Next, we can define the loss function using PyTorch's built-in `nn.MSELoss` module, which implements the Mean Squared Error loss function.

Python Code: Loss Function

```
1 # Define the Mean Squared Error loss function using nn.MSELoss
2 loss = nn.MSELoss()
```

Finally, we can define the optimization algorithm using PyTorch's `torch.optim.SGD` module, which implements the Stochastic Gradient Descent optimization algorithm.

Python Code: Optimizer

```
1 # Define the SGD optimization algorithm using torch.optim.SGD
2 trainer = torch.optim.SGD(net.parameters(), lr=0.03)
```

Now we can train the linear regression model using the defined components. The training loop remains similar to the previous implementation, but we can leverage the built-in modules to simplify the code.

Python Code: Training Loop

```
1 # Training loop
2 num_epochs = 3
3 for epoch in range(num_epochs):
4     for X, y in data_iter:
5         l = loss(net(X), y)
6         trainer.zero_grad()
7         l.backward()
8         trainer.step()
9     l = loss(net(features), labels)
10    print(f'epoch {epoch + 1}, loss {l:f}')
```

After training the model, we can evaluate its performance by comparing the learned parameters with the true parameters used to generate the synthetic data.

Python Code: Model Evaluation

```

1 w = net.weight.data
2 b = net.bias.data

```

2.4 Softmax Regression

Softmax regression, also known as multinomial logistic regression, is an extension of linear regression that is used for multi-class classification problems. Unlike linear regression, which predicts continuous output values, softmax regression predicts the probabilities of each class for a given input.

2.4.1 Model Definition

The output of the softmax regression model is a probability distribution of each class. Considering a picture classification task with k classes, the output of the model is a k -dimensional vector, where each element represents the predicted probability of the input belonging to that class. The sum of all probabilities in the output vector is equal to 1.

So here comes the first question, how to choose the labels for multi-class classification? The answer is to use **one-hot encoding**.

Definition 2.4.1

One-hot encoding is a technique used to represent categorical variables as binary vectors. In one-hot encoding, each category is represented by a vector of length equal to the number of categories, where all elements are 0 except for the element corresponding to the category, which is set to 1.

After defining the labels, we can define the softmax regression model. The model can be expressed as follows:

$$o_1 = x_1w_{11} + x_2w_{12} + x_3w_{13} + x_4w_{14} + b_1$$

$$o_2 = x_1w_{21} + x_2w_{22} + x_3w_{23} + x_4w_{24} + b_2$$

$$o_3 = x_1w_{31} + x_2w_{32} + x_3w_{33} + x_4w_{34} + b_3$$

where o_i is the output score for class i , x_j is the input feature j , w_{ij} is the weight connecting input feature j to class i , and b_i is the bias term for class i .

We can use a picture to illustrate the model structure:

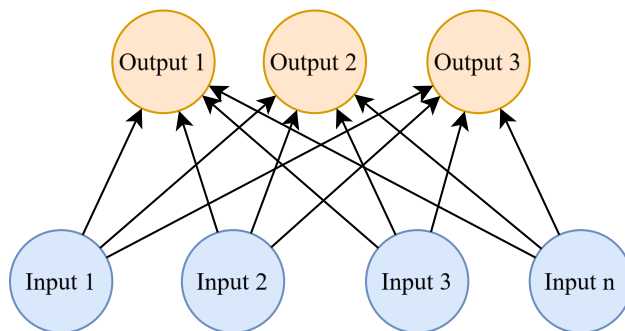


Figure 2.2: Softmax Regression Model Structure

Now we have the output scores for each class, but we need to convert these scores into probabilities. This is where the softmax function comes into play.

We cannot directly use the output scores for classification, as they can be any real number and do not represent probabilities. The softmax function transforms the output scores into probabilities by applying the following formula:

$$P(y = i|x) = \frac{e^{o_i}}{\sum_{j=1}^k e^{o_j}}$$

where $P(y = i|x)$ is the predicted probability of class i given input x , o_i is the output score for class i , and k is the total number of classes.

Although the softmax function is not linear, the model is still called softmax regression because the output scores o_i are computed using a linear combination of the input features. Thus, softmax regression can be seen as a linear model followed by a non-linear activation function (softmax) to produce probabilities.

We also need to define the loss function for softmax regression. The most commonly used loss function for multi-class classification is the Cross-Entropy Loss, which measures the difference between the predicted probability distribution and the true distribution (one-hot encoded labels).

The Cross-Entropy Loss can be expressed as follows:

$$L = - \sum_{i=1}^k y_i \ln(P(y = i|x))$$

We can derive the loss function from the likelihood function, just like what we have done in linear regression. Given a training dataset with m examples, the likelihood function for the entire dataset can be expressed as the product of the individual likelihoods for each training example:

$$L(w, b) = \prod_{i=1}^m P(y^{(i)}|x^{(i)}; w, b)$$

Taking the natural logarithm of the likelihood function, we have:

$$\ln L(w, b) = \sum_{i=1}^m \ln(P(y^{(i)}|x^{(i)}; w, b))$$

Maximizing the log-likelihood function is equivalent to minimizing the negative log-likelihood. Ignoring the constant terms, we can focus on minimizing the following expression:

$$- \sum_{i=1}^m \sum_{j=1}^k y_j^{(i)} \ln(P(y = j|x^{(i)}; w, b))$$

This is exactly the Cross-Entropy Loss function we defined earlier for softmax regression. Therefore, by maximizing the likelihood of the observed data given the model parameters, we arrive at the conclusion that minimizing the Cross-Entropy Loss function is equivalent to maximizing the likelihood.

For how to derive the Cross-Entropy Loss function in detail, please refer to the content about **Information Theory** on the internet.

2.4.2 MNIST

The MNIST dataset is a widely used benchmark dataset for image classification tasks, particularly for handwritten digit recognition. It consists of 60,000 training images and 10,000 test images of handwritten digits (0-9), each represented as a 28x28 grayscale image.

In this section, we will implement softmax regression using PyTorch to classify the Fashion-MNIST dataset. We will follow similar steps as in the previous linear regression example, but with modifications to accommodate the multi-class classification task.

Python Code: Libraries

```
1 import torch
2 import torchvision
3 from torch.utils import data
4 from torchvision import transforms
```

Now we will load the Fashion-MNIST dataset using PyTorch's `torchvision` module. We will apply necessary transformations to the images, such as converting them to tensors and normalizing the pixel values.

Python Code: Data Loading

```
1 # Load the Fashion-MNIST dataset
2 trans = transforms.ToTensor()
3 mnist_train = torchvision.datasets.FashionMNIST(
4     root="./data", train=True, transform=trans, download=True)
5 mnist_test = torchvision.datasets.FashionMNIST(
6     root="./data", train=False, transform=trans, download=True)
```

As we have already known, the **Fashion-MNIST** dataset contains 10 classes of clothing items, including T-shirts, trousers, pullovers, dresses, coats, sandals, shirts, sneakers, bags, and ankle boots. Each image is a 28x28 grayscale image, which we will flatten into a 784-dimensional vector for input to the softmax regression model.

Now let's define the data loader to read the data in minibatches.

Python Code: DataLoader

```
1 batch_size = 256
2 train_iter = data.DataLoader(mnist_train, batch_size, shuffle=True, num_workers
3                             =4)
```

Now we can define the function `load_data_fashion_mnist` to encapsulate the data loading process for convenience.

Python Code: Data Loading Function

```

1 def load_data_fashion_mnist(batch_size, resize=None):
2     trans = [transforms.ToTensor()]
3     if resize:
4         trans.insert(0, transforms.Resize(resize))
5     trans = transforms.Compose(trans)
6     mnist_train = torchvision.datasets.FashionMNIST(
7         root="../data", train=True, transform=trans, download=True)
8     mnist_test = torchvision.datasets.FashionMNIST(
9         root="../data", train=False, transform=trans, download=True)
10    return (data.DataLoader(mnist_train, batch_size, shuffle=True,
11                            num_workers=get_dataloader_workers()),
12            data.DataLoader(mnist_test, batch_size, shuffle=False,
13                            num_workers=get_dataloader_workers()))

```

The parameter "resize" allows us to resize the images if needed. The function returns the training and test data loaders.

Now we can define the softmax regression model using PyTorch's `nn.Linear` module, which automatically handles the initialization of weights and biases for us.

Firstly let's initialize the model parameters (weights and bias):

Python Code: Model Definition

```

1 num_inputs = 784
2 num_outputs = 10
3
4 W = torch.normal(0, 0.01, size=(num_inputs, num_outputs), requires_grad=True)
5 b = torch.zeros(num_outputs, requires_grad=True)

```

Then we can define the softmax regression model, loss function, and optimization algorithm.

Python Code: Main Components

```

1 # Define the softmax regression model
2 def softmax(X):
3     X_exp = torch.exp(X)
4     partition = X_exp.sum(1, keepdim=True)
5     return X_exp / partition

```

During the process of calculating the softmax function, we use the `keepdim=True` parameter to ensure that the output tensor maintains the same number of dimensions as the input tensor. This is important for subsequent operations, such as loss calculation, where we need to align the dimensions of the predicted probabilities and the true labels. Also, we use the **broadcasting** feature of PyTorch to perform element-wise division between the exponentiated scores and the partition function.

Python Code: Loss Function and Optimizer

```

1 # Define the cross-entropy loss function
2 def net(X):
3     return softmax(torch.matmul(X.reshape((-1, W.shape[0])), W) + b)
4
5 # Define the cross-entropy loss function
6 def cross_entropy(y_hat, y):
7     return -torch.log(y_hat[range(len(y_hat))], y])

```

Instead of computing the loss using the full formula that sums over all classes, this implementation uses an efficient indexing trick. The expression `y_hat[range(len(y_hat))]` performs advanced indexing:

1. For each sample in the batch, it selects the predicted probability corresponding to the true class label y .
2. This results in a 1-dimensional tensor containing the predicted probabilities for the correct classes for all samples in the batch.
3. Finally, the negative logarithm of these probabilities is computed to obtain the cross-entropy loss for each sample.

It is logically equivalent to the function below:

Python Code:

```

1 def cross_entropy_onehot(y_hat, y):
2     y_onehot = torch.zeros_like(y_hat)
3     y_onehot[range(len(y_hat)), y] = 1
4     return -(y_onehot * torch.log(y_hat)).sum(dim=1)

```

But the first implementation is more efficient in terms of computation and memory usage.

Afterall, we have defined the loss function. However, in real life applications, we need to output the predicted class labels based on the predicted probabilities. This can be achieved using the `argmax` function, which returns the index of the maximum value along a specified dimension.

Python Code: Prediction Function

```

1 def accuracy(y_hat, y):
2     if len(y_hat.shape) > 1 and y_hat.shape[1] > 1:
3         y_hat = y_hat.argmax(axis=1)
4     cmp = y_hat.type(y.dtype) == y
5     return float(cmp.type(y.dtype).sum())

```

Here we define a class `Accumulator` to help us accumulate the sum of multiple variables over time, which is useful for tracking metrics like loss and accuracy during training.

Python Code: Accumulator Class

```

1 class Accumulator:
2     def __init__(self, n):
3         self.data = [0.0] * n
4
5     def add(self, *args):
6         self.data = [a + float(b) for a, b in zip(self.data, args)]
7
8     def reset(self):
9         self.data = [0.0] * len(self.data)
10
11     def __getitem__(self, idx):
12         return self.data[idx]

```

Finally, we can define the training function to encapsulate the training process for convenience.

Python Code: Training Function

```

1 def train_epoch_ch3(net, train_iter, loss, updater):
2     if isinstance(net, torch.nn.Module):
3         net.train()
4     metric = Accumulator(3)
5     for X, y in train_iter:
6         y_hat = net(X)
7         l = loss(y_hat, y).sum()
8         if isinstance(updater, torch.optim.Optimizer):
9             updater.zero_grad()
10            l.backward()
11            updater.step()
12        else:
13            l.backward()
14            updater(X.shape[0])
15        metric.add(float(l), accuracy(y_hat, y), y.numel())
16    return metric[0] / metric[2], metric[1] / metric[2]

```

The training function defined above trains the model for only one epoch. To evaluate the model's performance on the test dataset, we need a function to compute the accuracy on the test set.

Python Code: Evaluation Function

```

1 def evaluate_accuracy(net, data_iter):
2     if isinstance(net, torch.nn.Module):
3         net.eval()
4     metric = Accumulator(2)
5     with torch.no_grad():
6         for X, y in data_iter:
7             metric.add(accuracy(net(X), y), y.numel())
8     return metric[0] / metric[1]

```

Now we can combine everything to define the complete training loop, which iterates through multiple epochs, trains the model, and evaluates its performance on the test dataset at the end of each epoch.

Python Code: Complete Training Loop

```

1 def train_ch3(net, train_iter, test_iter, loss, num_epochs, updater):
2     for epoch in range(num_epochs):
3         train_metrics = train_epoch_ch3(net, train_iter, loss, updater)
4         test_acc = evaluate_accuracy(net, test_iter)
5         print(f'epoch {epoch + 1}, loss {train_metrics[0]:.4f}, '
6               f'train acc {train_metrics[1]:.3f}, test acc {test_acc:.3f}')

```

With all the components in place, we can now train the model.

Python Code: Run Training

```

1 num_epochs = 10
2 lr = 0.1
3
4 def updater(batch_size):
5     return torch.optim.SGD([W, b], lr=lr)
6
7 train_ch3(net, train_iter, mnist_test, cross_entropy, num_epochs, updater)

```

Before making predictions, we need to define some helper functions to visualize the images and labels. We also need to import `matplotlib.pyplot`.

Python Code: Visualization Helpers

```

1 import matplotlib.pyplot as plt
2
3 def get_fashion_mnist_labels(labels):
4     text_labels = ['t-shirt', 'trouser', 'pullover', 'dress', 'coat',
5                   'sandal', 'shirt', 'sneaker', 'bag', 'ankle boot']
6     return [text_labels[int(i)] for i in labels]
7
8 def show_images(imgs, num_rows, num_cols, titles=None, scale=1.5):
9     figsize = (num_cols * scale, num_rows * scale)
10    _, axes = plt.subplots(num_rows, num_cols, figsize=figsize)
11    axes = axes.flatten()
12    for i, (ax, img) in enumerate(zip(axes, imgs)):
13        if torch.is_tensor(img):
14            ax.imshow(img.numpy())
15        else:
16            ax.imshow(img)
17            ax.axes.get_xaxis().set_visible(False)
18            ax.axes.get_yaxis().set_visible(False)
19        if titles:
20            ax.set_title(titles[i])
21    return axes

```

After training, we can use the trained model to make predictions on some images from the test dataset.

Python Code: Prediction

```
1 def predict_ch3(net, test_iter, n=6):
2     for X, y in test_iter:
3         break
4         trues = get_fashion_mnist_labels(y)
5         preds = get_fashion_mnist_labels(net(X).argmax(axis=1))
6         titles = [true + '\n' + pred for true, pred in zip(trues, preds)]
7         show_images(X[0:n].reshape((n, 28, 28)), 1, n, titles=titles[0:n])
8
9 predict_ch3(net, mnist_test)
```

This concludes the implementation of Softmax Regression using PyTorch. The model is now capable of classifying images from the Fashion-MNIST dataset with reasonable accuracy.

You can run the complete code in a Python environment with PyTorch and torchvision installed to see the results. We provide the complete code in the folder for your convenience.

Now you have learned how to implement both Linear Regression and Softmax Regression using PyTorch. These foundational techniques are essential for understanding more complex machine learning models and algorithms that will be covered in later chapters.

In the next chapter, we will explore a real neural network architecture called **Multilayer Perceptron (MLP)**, which builds upon the concepts learned in this chapter and introduces additional layers and non-linear activation functions to enhance the model's capacity to learn complex patterns in data.

Chapter 3

Multilayer Perceptrons

In this chapter, we will explore the first type of neural network architecture: the multilayer perceptron (MLP). MLPs are a class of feedforward artificial neural networks that consist of multiple layers of nodes, or neurons, each fully connected to the next layer. They are widely used for various machine learning tasks, including classification and regression.